EKATRA

Project scope & Statement of Work

Version 1.4 | 16 September 2026

What this project is

Ekatra is a messaging-led learning platform. It helps organizations deliver short, structured courses through the channels learners already use, while administrators manage courses, learners, progress and delivery issues in one web console.

Why it is being built

A messaging-learning service must be dependable before it is used with real learners. Messages may repeat, providers may reject sends, workers may restart and organizations must remain isolated from one another. This project makes the existing platform safe, supportable and ready to operate.

Stage 1 - WhatsApp learning platform

The launch scope: WhatsApp learner delivery, React administration console, and the API, worker, database, Redis, security, scheduling and operational controls needed to run them. Target: less than four calendar weeks from an unblocked kickoff.

Stage 2 - Full multi-channel product

Future milestones, estimated separately: Telegram delivery; learner web; offline mobile and synchronization; direct Meta Cloud API and GramIQ WhatsApp adapters.

Starting point for a developer

Published code: github.com/iabheejit/ekatra-platform (main)
Local working directory: /Users/Shared/Scripts/ekatra-master
Canonical application repository: active/ekatra-platform
The repository already contains roughly 25-30% of the Stage 1 foundations, including build, configuration, migration, readiness and selected admin-workflow work. It must be reviewed and extended, not treated as a production-accepted release.

Stage 1 acceptance journey

Admin publishes the representative course -> enrolls a learner -> schedules delivery -> learner receives and completes WhatsApp lessons -> pause/opt-out suppresses future sends -> progress survives restart and duplicate callback -> failures appear in admin -> one certificate record exists.

Contents

Part I - Project scope, delivery stages and developer starting point
Part II - Statement of Work, work packages and acceptance gates

Source documents: PRODUCTION_SCOPE.md and DEVELOPER_SOW.md.
Detailed implementation evidence: HARDENING_PROGRESS.md.
The following parts reproduce the updated project scope and SOW in full.

Ekatra production scope

Version 1.4 · 16 September 2026 · New-developer project brief; published code repository; four-week Stage 1 target

Project scope

Ekatra is a learning platform that delivers short, structured courses through messaging channels. It exists to let organizations run practical learning programmes where learners already communicate, while giving programme teams one place to create courses, enroll learners, schedule delivery, monitor progress and respond to delivery problems.

The project is being built in two stages.

1. **Stage 1 - WhatsApp learning platform:** a working WhatsApp learning service and React administration console, supported by an API, background worker, PostgreSQL, Redis, security, scheduling and operational controls. This is the launch scope and targets delivery in less than four weeks from an unblocked kickoff.

2. **Stage 2 - Full multi-channel product:** additional learner channels and experiences, delivered as separate milestones after Stage 1. It includes Telegram, learner web, offline mobile, direct Meta Cloud API and GramIQ WhatsApp support.

Stage 1 uses the WhatsApp provider currently serving customer usage unless discovery identifies a release blocker. The React admin is part of Stage 1 because the learning operation cannot be run safely without course, learner, progress and failure-management tools. The four-week target applies only to Stage 1. Stage 2 is a roadmap, not part of the Stage 1 acceptance commitment.

Why this work is needed

The repository already contains the main product components, but a messaging-learning system must be dependable before it can be used with real learners. A message can arrive twice, a provider can reject a send, a worker can restart mid-course, and different organizations must never see one another's learners or courses. Stage 1 turns the existing product into a controlled, supportable service by making those conditions safe and observable.

The purpose is not to replace the platform. It is to complete the core learning journey, make its boundaries clear, and give a new developer a practical baseline on which to work.

What Stage 1 must deliver

Finish the WhatsApp authored-course journey in the existing React admin before adding product surfaces. Priorities are organization/role access, durable and replay-safe delivery, correct scheduling, safe course editing, truthful UI states and operational recovery. AI assistance must remain optional to deterministic delivery. Telegram, learner web, offline mobile and additional providers remain Stage 2 work.

Stage 1 acceptance journey

In staging, an authorized admin publishes the approved representative course, enrolls an authorized learner and schedules delivery. The learner receives and completes the course through the selected WhatsApp provider; pause and opt-out suppress future sends; progress survives a worker restart and a duplicate provider callback; delivery failures appear in the React admin; and completion produces one consistent certificate record. The provider, hosting target and approved pilot cohort must be recorded before this journey begins.

An authorized admin can publish an approved course, enroll a learner, schedule delivery, see progress and message failures, and recover failed work. A consenting learner can receive lessons/media, answer quizzes, ask a bounded course question, pause/resume, opt out, finish a course and receive a certificate. Progress survives process restarts and repeated provider callbacks. Organizations cannot access one another's data.

Release acceptance centers on one representative authored course; the implementation must support additional courses through data/configuration without bespoke code. Preserve existing required features used by current customers, identified during discovery.

Developer starting point

The published code repository is github.com/iabheejit/ekatra-platform. The main branch is the shared starting point for a new developer. Project scope, SOW, implementation evidence, assessment and PDF are available in docs/project.

For local development, the working directory is /Users/Shared/Scripts/ekatra-master and the canonical application repository is /Users/Shared/Scripts/ekatra-master/active/ekatra-platform. A developer should work on top of the published main branch, preserve existing customer behaviour and retain the documented release gates.

The current working copy contains approximately **25-30% of the Stage 1 implementation foundations**. This is an engineering estimate, not a delivery-completion percentage, a payment milestone or production acceptance. It includes clean-build, configuration, migration, shared delivery-code, readiness and selected admin-workflow improvements. The remaining security, reliable delivery, scheduling, operational recovery and end-to-end acceptance work is still part of Stage 1.

- **Most build foundations implemented locally:** clean builds, shared delivery code, aligned configuration, migration runner and deployable package assembly. Container/staging and full-schema migration verification remain.

- **Security and reliability started:** admin token validation, verified webhook signatures, worker/Redis setup and failure reporting. Tenant isolation, durable receipt, safe retries, scheduling and learner suppression remain.

- **Admin workflow repairs started:** typed API mapping and course edits that retain existing lesson IDs. Atomic saves, server-side deletion safeguards, full journey validation and unsupported-control cleanup remain.

- **Operations foundations started:** CI configuration, readiness and documentation. Alerts, capacity, backup/restore, rollback and production acceptance remain.

Evidence recorded to date: **46 passing unit/regression tests**, one separately verified migration integration test and a clean-source build; the latest editor follow-up also passes frontend TypeScript validation. These are cumulative local checks, not hosted CI or production acceptance. Review and validate the existing foundations before extending them. See DEVELOPER_SOW.md for the work packages and HARDENING_PROGRESS.md for detailed evidence.

Remaining capabilities: fund these first

Missing capability	Production consequence	Required response
Reproducible builds and aligned configuration	A checkout cannot reliably become a release	Fix backend/worker builds, dependencies, migration execution and deployment configuration
Consistent tenant/role isolation	Authenticated users can access inappropriate data	Derive tenant from verified identity; apply ownership checks to every affected API operation
Verified ingress and durable delivery	Forged, repeated or interrupted events can damage progress	Authenticate enabled webhooks, persist before acknowledgment, make processing replay-safe
Correct scheduler	Timed lessons can be missed or duplicated	Evaluate due timestamps, respect timezone, recover missed work and record delivery outcomes
Operational visibility and recovery	Failures remain silent or require database editing	Dependency readiness, queue/error metrics, alerts, bounded retry/replay, backup/restore and rollback
Supported admin workflows	UI presence does not guarantee correct behavior	Validate course editing/publishing, enrollment, message history, progress and failed-delivery handling
Learner controls and data lifecycle	Operational team cannot handle stop/delete/export requests consistently	Persist opt-out, suppress scheduled sends, document retention and provide controlled export/deletion procedures
Measured acceptance and capacity	“Production ready” is subjective	Automated lifecycle/failure tests plus measured pilot load and runbook evidence

Missing capability	Production consequence	Required response
Ownership and deployment inventory	Nobody knows which docs or environment represent reality	Record deployed SHA, environment, migration state, owners and supported channel

Overdone: simplify these

Area	Scope correction
Competing Azure/Railway/AWS deployment narratives	Maintain one production deployment path and one local development path
Multiple provider/compatibility routes	Enable only the selected provider; preserve inactive adapters behind explicit configuration where useful
Multiple admin frontends	Choose the existing React admin as the maintained interface; retire static admin only after checking actual usage and replacement parity
Cross-app worker imports	Share only the delivery/domain code needed by API and worker through a supported package; avoid new microservices
Redis optional mode alongside production queues	Require Redis for production delivery; explicit degraded local development mode may remain
Several AI providers/tiering/media helpers	Use the existing working text configuration with explicit timeouts, budget limits and fallback; hide unsupported capabilities
Broad visual FSM authoring ambition	Keep authored-course import/validation and necessary editing; do not redesign the graph editor
Dashboard breadth without trustworthy data	Keep course, learner, progress, delivery and error views; hide incomplete or misleading panels
Tracked generated files and environment drift	Agree source/artifact policy; produce deployable artifacts in CI; avoid committing unrelated build churn

“Overdone” describes the scope and maintenance burden, not a claim that every advanced feature is fully implemented. Do not delete working customer functionality based only on this table.

Stage 2 - Full multi-channel product: separately estimated milestones

- Stage 2A: Telegram learner delivery, including bot setup, webhook verification and channel acceptance tests.

- Stage 2B: learner web launch, including production OTP authentication, course consumption, quizzes, tutor interaction and progress.

- Stage 2C: offline mobile application release and platform synchronization, with a separate device, conflict-resolution and offline acceptance plan.

- Stage 2D: direct Meta Cloud API and GramIQ WhatsApp adapters. The first release may retain the provider already used by customers; each additional adapter becomes supported only after end-to-end provider validation.

Each Stage 2 milestone requires its own estimate, dependencies, acceptance fixture and release decision. Stage 2 is a roadmap heading, not a single delivery commitment.

These items are product commitments in the larger scope, but are not acceptance requirements for the minor four-week release.

Outside both current scopes: defer explicitly

- New multi-agent orchestration, knowledge graphs, mastery prediction and adaptive curriculum research.

- Semantic caching targets or elaborate routing without observed cost/latency need.

- Voice/IVR, image understanding, AI-generated lesson media and video generation.

- WATI compatibility expansion or additional messaging providers beyond Meta Cloud API and GramIQ.

- Maya/Asmaa consolidation or edge hub deployment beyond the agreed offline mobile application and synchronization milestone.

- Autonomous marketing/sales/company agents in ekatra-learning.

- Legacy rewrites, bulk archive cleanup and migrations of all historical courses/users.

- Billing/subscriptions, self-service tenant signup and enterprise SSO.

- New spaced-repetition/FSRS UX or optimization; preserve an already-used reminder flow only if documented as required during discovery.

- A claim of 10,000 concurrent users, specific completion improvement, or specific cost per learner without evidence.

Offline mobile is part of the larger product scope, but requires a separate milestone estimate and device acceptance plan. It must not inflate or delay the minor four-week release.

Required safeguards when narrowing scope

1. Inventory actual customer dependencies before disabling routes or UI.

2. Default unsupported features off; show only supported actions in production.

3. Preserve data and compatibility needed for existing users. Provide a transition plan for any removed surface.

4. Record the provider currently used by customers during discovery. The minor release keeps that provider unless it is blocked. Meta Cloud API and GramIQ are separately enabled and accepted under the larger scope.

5. Treat certificates as completion records; do not introduce mastery/accreditation claims.

6. Keep AI assistance bounded to available course content and an explicit fallback; AI failure must not stop deterministic lesson delivery.

Delivery and definition of done

See HARDENING_PROGRESS.md for completed implementation, test evidence and remaining developer work. The release gates remain in force as implementation progresses.

Use DEVELOPER_SOW.md for work packages, acceptance tests, estimate assumptions and handover. Use the assessment for source evidence. Historical plans are references, not additional requirements.

Statement of Work - Ekatra production hardening

Version: 1.4 **Prepared:** 15 September 2026 **Status:** Stage 1 - WhatsApp learning platform confirmed for less than four weeks; Stage 2 - Full multi-channel product recorded as separately estimated milestones **Client:** Ekatra **Developer/vendor:** To be assigned **Primary repository:** active/ekatra-platform

1. Project description and objective

Ekatra is a messaging-led learning platform. Organizations use it to turn an approved course into scheduled, short learning interactions, while administrators manage courses, learners, progress and delivery issues in one web console. Learners receive course content and respond through their messaging channel rather than needing to adopt a new learning application.

The immediate objective is to deliver a reproducible, secure and supportable Stage 1 WhatsApp learning platform. It must allow an authorized administrator to publish and operate a course, and an authorized learner to receive, complete, pause or opt out of that course without lost progress or hidden delivery failure.

This is a focused hardening engagement, not a platform rewrite. The existing TypeScript, React, PostgreSQL, Redis and worker-based architecture remains the baseline. The project exists because messaging delivery, learner data and course progression need production controls: access isolation, verified provider ingress, safe retries, correct scheduling, visible failures and recovery procedures.

Stage 1 must be completed in less than four calendar weeks from an unblocked kickoff. It covers WhatsApp learner delivery, the web administration console and the platform services needed to operate them. Telegram, learner web, offline mobile and additional WhatsApp-provider certification are Stage 2 work and do not form part of the Stage 1 acceptance gate.

Developer starting point

The published code repository is github.com/iabheejit/ekatra-platform. Its main branch is the shared starting point for the engagement; the project brief, scope, SOW, evidence, assessment and PDF are in docs/project. Work should extend and validate this baseline rather than rebuild it or remove existing customer behaviour without an approved transition.

For local development, the working directory is /Users/Shared/Scripts/ekatra-master; the canonical application repository is /Users/Shared/Scripts/ekatra-master/active/ekatra-platform.

The current working copy includes approximately **25-30% of the Stage 1 implementation foundations**: clean-build improvements, shared delivery code, configuration and migration safety, readiness checks, and selected admin workflow repairs. It also contains **46 passing unit/regression tests**, one isolated migration integration test and a clean-source build result. This evidence is useful starting material, but it is not staging or production acceptance. The remaining Stage 1 responsibilities are set out in this SOW and in HARDENING_PROGRESS.md.

The baseline is PRODUCTION_SCOPE.md, supported by the repository assessment. Earlier architecture/execution plans do not add obligations to this SOW.

2. Product boundary and assumptions

Stage 1 - WhatsApp learning platform: WhatsApp learner delivery, the React admin console, and the API, worker, PostgreSQL, Redis, security, scheduling and operational controls necessary to support them; target less than four calendar weeks from an unblocked kickoff.

Stage 2 - Full multi-channel product: the complete product across WhatsApp, Telegram, learner web and the offline mobile application, including direct Meta Cloud API and GramIQ WhatsApp support. Stage 2 is split into separately estimated milestones: Telegram delivery, learner web, offline mobile and each additional WhatsApp adapter. Throughout this document, "minor release" means Stage 1 and "larger follow-on scope" means Stage 2.

Included

- Existing TypeScript backend, React admin UI, PostgreSQL database, messaging package and Redis-backed worker.

- One active WhatsApp provider; preserve the provider used by current customers unless discovery establishes a blocker.

- One cloud hosting target; existing working production host preferred.

- Isolated staging and production configurations, with one documented local development path.

- Admin-managed courses and enrollments; tenant and role enforcement for existing organizations.

- Authored lesson/media delivery, quizzes, progress, pause/resume, opt-out, bounded course Q&A, completion and certificates.

- Scheduling, message history/status, failure handling and operational monitoring.

- Hardening existing AI generation if currently exposed; generated content must remain draft until an authorized admin publishes it.

- Import and acceptance of one client-supplied representative course, including media and quiz content.

Excluded

The four-week minor release excludes Telegram launch, learner web/OTP launch, offline mobile deployment and synchronization, and certification of an additional WhatsApp provider. It also excludes new multi-agent systems, adaptive curriculum, voice/video/image generation, advanced RAG/semantic caching, new FSRS experiences, billing, self-service signup, new SSO, large-scale historical migrations, content authoring/translation, redesign of the visual FSM editor and infrastructure vendor migration unless necessary and separately accepted.

Stage 2 - Full multi-channel product

The larger scope comprises WhatsApp, Telegram, learner web and the offline mobile application. Its target WhatsApp integrations are direct Meta Cloud API and GramIQ WhatsApp. Each provider must have isolated configuration, verified ingress, delivery-status handling and an end-to-end acceptance journey. One provider is active per environment.

Stage 2 has four separately estimated milestones: (A) Telegram learner delivery, (B) learner web, (C) offline mobile and synchronization, and (D) direct Meta Cloud API and GramIQ WhatsApp adapter certification. Each needs its own dependencies, acceptance fixture, estimate and release decision after the Stage 1 baseline is verified. Stage 2 does not inherit the four-week deadline.

Existing customer dependence on an excluded capability must be identified in discovery. It is not authorization to remove that capability. The client decides whether to preserve it within a revised estimate or agree a transition.

Proposed sizing assumptions, not existing performance claims

Estimate against 1,000 enrolled learners, 100 simultaneously active learner conversations, and a burst of 10 inbound webhook requests/second for 10 minutes with provider/AI calls stubbed during load testing. Developer must confirm realism against actual traffic and provider constraints during discovery. Real provider tests use a small approved test cohort.

No uptime SLA or learner-outcome improvement is implied. Agree operational targets before implementation; proposed recovery targets are RPO ≤24 hours and RTO ≤4 hours, verified by restore rehearsal. Hosting/provider fees and business-side approvals are client dependencies.

3. Work packages

Implementation status and verification evidence are maintained in HARDENING_PROGRESS.md. Estimate remaining work against that record; completed source changes do not replace staging or production acceptance requirements.

Implementation coverage snapshot - 15 September 2026

Approximately **25-30% of the minor SOW implementation is covered**, based on engineering judgment of the implemented foundations and remaining work. This is not a measured task-weighted percentage, a payment milestone, a

production-readiness score or a claim that 25-30% of delivery time has been saved. No complete work package has passed its full SOW acceptance gate. The larger follow-on scope is excluded from this estimate.

Work package	Coverage to date	Remaining developer responsibility
WP0 - Scope/discovery	Minor release boundary agreed; repository assessed	Verify deployed SHA, host/provider, operational owners, customer dependencies and acceptance fixture
WP1 - Build/deployment	Most foundations implemented locally: clean build, shared delivery package, configuration, migration runner and packaging	Review source; exercise containers/staging; validate full-schema migration and existing-database baseline; complete secret/artifact policy checks
WP2 - Security	Partial: admin token checks, signed webhook ingress and protected health actions	Complete tenant/role isolation, provisioning/revocation, disabled-route policy, opt-out and data controls
WP3 - Delivery/recovery	Foundation: worker initialization, Redis configuration and failed-send reporting	Durable inbound receipt, duplicate/concurrency safety, outbound attempts, correct scheduling and controlled replay
WP4 - Product journey	Partial: API/UI contracts and mapping, type fixes and lesson-ID-preserving course edits	Transactional/concurrency-aware saves, server-side deletion safeguards, full learner journey, certificates, bounded AI and unsupported-control cleanup
WP5 - Operations	Foundation: CI configuration, dependency readiness and setup documentation	Hosted CI/lint, tested artifact deployment, monitoring/alerts, load tests, restore/rollback, live acceptance and handover

Recorded evidence comprises **46 passing unit/regression tests**, one separately verified migration integration test and a clean-source build. The latest course-editor follow-up passed all six frontend tests and frontend TypeScript checks. These are cumulative local results, not a single hosted acceptance run. The migration test used an isolated PostgreSQL cluster; full application-schema acceptance remains open. No production deployment or real-learner verification has been performed.

Effect on the remaining engagement: review, test and accept the implemented foundations instead of estimating their implementation again from scratch. Retain all staging, security, reliability and recovery gates. Re-estimate remaining effort after validating these changes and production assumptions; no fee or deadline adjustment is implied by this snapshot. Delivery correctness, organization isolation and production recovery remain the principal implementation risks.

WP0 - Verify reality and freeze release scope

Tasks

- Record repository/branch state, deployed SHA, active URLs, host, provider, database/migration state, worker topology and operational owners.

- Inspect existing local changes and preserve them; identify which changes belong to this release.

- Establish a supported-feature matrix from current customer usage. Record the active WhatsApp provider and deployment target; preserve working customer behavior.

- Confirm course fixture, pilot cohort, languages already supported, roles, completion rules, traffic baseline, retention period and recovery targets.

- Create release backlog with dependency order and estimates; identify required provider templates/credentials without exposing secrets.

Deliverables: deployment inventory; agreed feature matrix; decision log; ranked backlog; revised estimate.

Acceptance: client/product owner and developer agree the release channel, exclusions, acceptance fixture and operating targets. Unknown production state is explicitly recorded, not presented as verified.

WP1 - Make builds and deployment reproducible

Tasks

- Review and accept the implemented kickoffAuthored.ts fix and @ekatra/delivery extraction; both now pass the clean-source build. Verify the packaged API and worker in staging.

- Validate the corrected Docker packaging, runtime imports, provider variable names, JWT settings and frontend build-time API URL in the selected environment. Local production bundles have been assembled; containers remain to be exercised.

- Pin supported Node/pnpm versions consistently; use lockfile installs.

- Establish safe environment validation and secret handling; inspect tracked source/history for exposed secrets and coordinate rotation of confirmed exposures.

- Validate the implemented migration ledger/order/checksums and transaction failure behavior against the complete application schema. Reconcile a reviewed ledger baseline for existing databases and test upgrade on an isolated copy; synthetic PostgreSQL regression coverage is already passing.

- Define compiled-artifact policy and prevent generated files from polluting source.

Deliverables: passing clean build; usable local/staging setup; configuration reference; repeatable migrations.

Acceptance: from a clean checkout, documented commands install, test and build every shipped package; container API and worker start; UI reaches intended API; repeat migration run is safe; failed migration cannot silently pass; no production database is modified during verification.

WP2 - Close access and ingress gaps

Tasks

- Enforce tenant membership and required roles across courses, learners, schedules, analytics, users, FSMs, messages and certificate-related operations.

- Review token issuance/validation and admin provisioning/recovery. Prevent cross-use of learner/admin tokens and client-selected privilege escalation.

- Require valid signatures/secrets for enabled provider endpoints, using raw request bytes where required by the selected provider. Disable unused ingress explicitly.

- Keep learner-web routes disabled for this release; ensure mock OTP cannot become active in production. Document their unfinished status.

- Apply bounded payloads and appropriate rate limiting, redact tokens/credentials and unnecessary learner data in logs, and validate configurable origins.

- Add learner opt-out persistence and suppression checks to every scheduled/broadcast send path. Provide controlled export/deletion workflow and retention configuration/runbook.

Deliverables: authorization policy; automated negative tests; enabled-route inventory; learner data/control runbook.

Acceptance: test accounts in two organizations cannot read/write each other's resources; lower roles cannot perform restricted actions; invalid signatures rejected; disabled endpoints cannot trigger processing; production mock auth unavailable; opt-out prevents future scheduled sends without erasing required state unexpectedly.

WP3 - Make inbound processing and scheduling recoverable

Tasks

- Persist accepted inbound events before acknowledgment; process with durable retries and correlation IDs.

- Make learner progress updates safe under duplicate events, retries and concurrent messages; distinguish received/processing/completed/failed outcomes.

- Track outbound attempts and provider identifiers. Handle uncertain send outcomes explicitly; do not promise universal exactly-once external delivery.

- Fix custom schedule due-time evaluation, timezone handling, missed ticks and daily/custom overlap.

- Require production Redis/worker dependencies. Handle worker crashes and failed sends without marking failures successful.

- Provide capped retry/backoff, dead-letter inspection and controlled replay; define replay permissions and audit trail.

Deliverables: durable inbound/outbound processing; corrected scheduling; operational replay procedure/tool.

Acceptance: repeated inbound event advances state once; crash after acceptance recovers; concurrent events do not corrupt progress; 09:30 and timezone-boundary fixtures deliver at intended times; restart catches up according to documented policy; opt-out/paused learners are excluded; failed work remains visible and replayable.

WP4 - Finish the supported learner and admin journey

Tasks

- Verify create/import → validate → draft → publish → enroll → schedule → deliver → quiz → pause/resume → complete → certificate.

- Make the supported React admin's course, learner, progress, history/status and failure views use correct APIs; show accurate loading/error/empty states.

- Hide controls that are unsupported in the minor release, including learner web, Telegram and inactive provider controls. Retire static admin only after WP0 confirms replacement parity and usage.

- Ensure text, required image/document media and certificates are accessible through production URLs with suitable access/lifetime rules and no unintended learner data exposure.

- Keep course progression independent of AI availability. Bound Q&A context, timeout and per-user usage; record model usage where available and provide clear fallback.

- Preserve existing required behavior without introducing new agent frameworks or retrieval infrastructure.

Deliverables: working supported UI; canonical course fixture; lifecycle integration tests; short acceptance demo.

Acceptance: a new authorized learner completes the fixture on the chosen provider; progress survives restart; errors do not appear as success; one logical completion produces a consistent certificate record; publication requires admin action; AI timeout does not block subsequent lessons; unsupported controls are absent or clearly unavailable.

WP5 - Operate, validate and hand over

Tasks

- CI checks all shipped builds/typechecks, meaningful lint and unit/integration tests. Fix the current lint baseline; do not disable rules globally to obtain a green result.

- Produce one traceable deployable artifact and deploy that tested artifact to staging/production through the agreed process.

- Add dependency-aware readiness, worker heartbeat, queue depth/age, failed delivery, API error/latency and AI usage visibility. Configure actionable alerts with named recipients.

- Prove backup/restore and previous-version rollback; document schema compatibility constraints and maintenance steps.

- Run the agreed load profile in staging, failure/replay suite, and a small real-channel acceptance journey.

- Deliver operational/developer documentation and train the maintainer.

Deliverables: CI/release workflow; test report; monitoring/alert evidence; recovery evidence; handover pack.

Acceptance: all release gates below pass, client accepts staging evidence, production deployment is authorized, post-deploy smoke checks pass, and the designated maintainer can follow the runbook.

4. Release acceptance matrix

Stage 1 acceptance journey

The staging demonstration must show this complete sequence: **authorized admin publishes the representative course -> enrolls an authorized learner -> schedules delivery -> learner receives and completes lessons through the selected WhatsApp provider -> pause/opt-out suppresses future sends -> progress survives worker restart and a duplicate provider callback -> delivery failure is visible in the admin -> exactly one completion certificate record exists.** The selected provider, hosting target and approved pilot cohort must be recorded before the demonstration. This journey supplements, and does not replace, the detailed gates below.

Gate	Required evidence
Clean build	Successful CI from a fresh checkout including backend, worker, frontend and shared packages
Access isolation	Automated role/tenant denial tests plus valid-user positive cases
Ingress	Valid/invalid signature fixtures; inactive ingress rejection; event acknowledgment after durable receipt
Delivery recovery	Duplicate/restart/concurrency and failed-send replay tests with stable learner progress
Scheduling	Non-hour send time, timezone boundary, restart catch-up, overlap and opt-out test results
Learning journey	Recorded staging run of approved course and automated deterministic journey tests
AI fallback	Timeout/unavailable/limit cases; lessons still progress; generated content remains draft
Capacity	Agreed load profile completes; proposed webhook acknowledgment p95 ≤1s, zero lost accepted events, and queue drains within 5 minutes after the stubbed burst; report hardware and actual results
Operations	Readiness failure visible when required DB/Redis unavailable; worker-down alert triggers within proposed 5 minutes; named owner receives test alert
Recovery	Restore of isolated backup with measured RPO/RTO; rollback demonstration; migration compatibility notes
Production	Approved deploy, deployed SHA/artifact recorded, small real-provider smoke test, no unresolved release-blocking defects

Numbers in the capacity/operations rows are proposed acceptance targets, to be finalized in WP0. Full response latency with real AI/messaging providers is reported separately and must not be represented by stubbed-load results.

A release-blocking defect means unauthorized access, lost/corrupted learner progress, inability to complete the core journey, persistent delivery/scheduling failure, or inability to deploy/recover safely. Cosmetic defects may be accepted only in a documented residual backlog.

5. Stage 1 - WhatsApp learning platform: four-week delivery sequence

The minor scope has a fixed delivery target of less than four calendar weeks. It assumes one experienced full-time developer, immediate access to a usable staging environment and the active provider account, rapid product decisions, and an existing implementation that can be hardened rather than rebuilt. External access or approval delays move the acceptance date unless the parties explicitly reduce scope.

Period	Primary outcome	Work packages
Days 1-3	Reality verified, scope frozen, active provider/host recorded, clean-build blockers identified	WP0; WP1 starts
Week 1	Reproducible build, configuration, migrations and staging baseline	WP1; CI foundation
Week 2	Tenant/role enforcement, verified webhook ingress, opt-out and data controls	WP2; WP3 starts
Week 3	Durable processing, scheduling, retry/replay and core admin/learner journey	WP3; WP4
Final week	Acceptance suite, monitoring, restore/rollback evidence, production release and handover	WP4; WP5

WP0 must produce a task-level forecast within the first three working days. If verified effort exceeds the available four-week capacity, the product owner must defer non-critical breadth while retaining the security, data-isolation, progress-integrity, deployment and recovery gates. The larger follow-on scope receives its own estimates after this baseline.

Acceptance milestones are A = WP0-WP1 at the end of Week 1; B = WP2-WP3 at the end of Week 3; and C = WP4-WP5 before the four-week deadline. Fees, payment schedule, start date, review turnaround and any warranty/support period are to be agreed separately. Payment acceptance should refer to demonstrated deliverables, not time elapsed.

6. Responsibilities and dependencies

Client/product owner provides: environment/account access, provider test number and any required messaging templates, representative approved course/media, test users from two organizations, active-customer feature requirements, decisions on host/provider/retention/roles, staging acceptance and authorization for production release.

Developer provides: implementation, migration safety, tests/evidence, estimates and progress reporting, early escalation of blockers, release/rollback procedure and handover. Use test data and isolated resources for validation; never message real learners or migrate production incidentally during tests.

Operational owner provides/accepts: alert destination, backup ownership, incident triage and ongoing hosting/provider costs. Name this person before release.

Provider onboarding/approval, unavailable credentials, unknown production drift, unusable legacy data and missing course assets may affect the timeline. Record the blocked deliverable and unaffected work rather than silently assuming completion.

7. Change control and implementation constraints

- Preserve current customer behavior and existing local work until its ownership/scope is established.

- Submit reviewable changes by work package, with tests and rollout impact. Avoid unrelated refactors and dependency churn.

- Adding Telegram, learner web, offline mobile, Meta Cloud API or GramIQ certification to the minor release requires a written transfer from the larger scope with stated effort, schedule and acceptance impact.

- A new host, any additional provider/channel, broad historical migration or AI architecture expansion requires an explicit scope change containing effort, schedule and acceptance impact.

- Do not destructively clean archives, delete production data, rotate credentials or deploy without the relevant owner coordinating the action.

- Treat unavailable features as disabled capabilities, not dashboard promises.

- Do not perform production verification by sending unsanctioned messages or incurring unbounded AI spend.
- Maintain one current setup guide and one release runbook; mark conflicting historical plans superseded.

8. Handover checklist

- Merged/reviewed source and dependency lockfile; release tag/SHA and artifact reference.
- Supported-feature/channel matrix, environment variable names, local/staging setup and architecture notes.
- Migration ledger, deployment order, backup/restore and rollback instructions.
- Test fixtures, CI logs, load/failure report and real-provider acceptance evidence.
- Monitoring links, alert ownership, retry/replay and incident runbooks.
- Tenant/role policy, opt-out/export/delete/retention procedures and secret management instructions.
- Residual defects, deferred capabilities, operating costs observed during testing and ownership of follow-up work.
- Maintainer walkthrough and completed acceptance record.

9. Approval record

Product owner: _____ Developer: _____ Operational owner: _____

Stage 1 - WhatsApp learning platform: WhatsApp learner delivery, web admin and required operating platform; less than four weeks (confirmed 15 September 2026)

Stage 2 - Full multi-channel product: separately estimated Telegram, learner web, offline mobile and direct Meta Cloud API/GramIQ WhatsApp milestones

Selected provider: _____ Hosting target: _____ Pilot cohort: _____

Confirmed traffic/recovery targets: _____ Fee/start date/support terms: _____

Scope baseline accepted on: _____ Staging accepted on: _____ Production release accepted on: _____